Technische Universität Dresden

Data Visualization
Course Work

0 – Introduction & Toolkits

Faculty of Computer Science – Institute of Software and Multimedia Technology

Computergraphik und Visualisierung

INTERACTIVE MEDIA LAB DRESDEN





Lennart Woidtke

CGV Lab Dresden

Research Area

Realtime Rendering, Virtual Reality, Neural Rendering

Julián Méndez

Interactive Media Lab

Research Area

Interactive visualization for computational models

Course Work

(1) What to expect

TUD

Course Work
In a nutshell

5 assignments over the semester

Additional hands-on experience to topics discussed in the lecture:

- Working with common toolkits for developing visualizations
- Applying the principles taught in the lecture to example datasets

Exercise topics will be part of the exam!

- Please note: no bonus points!
- No transfer of results if you took the exercise in previous semesters.

Use of AI tools is allowed as long as you indicate how they were used.

Course Work
Schedule & Assignments

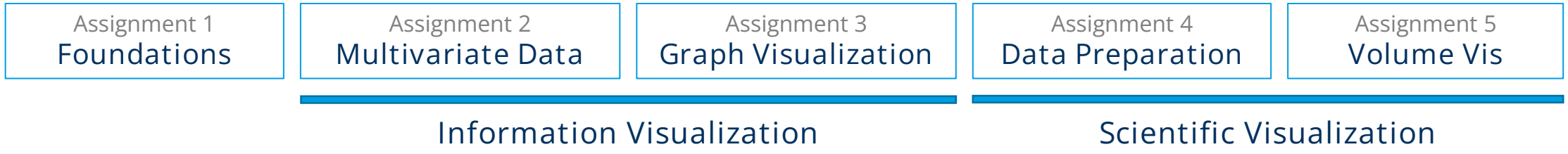
Five assignments over the semester

- Practical + theoretical tasks
- You work in teams of three (starting with A2)
- Presentation of selected results via video recording
- Feedback possibilities:

  During consultation slots (appointment by email)
- The exercises will not be reviewed, but you are encouraged to work on them!

This means…

- You will have to implement visualizations
- You should schedule enough time for working on the assignments



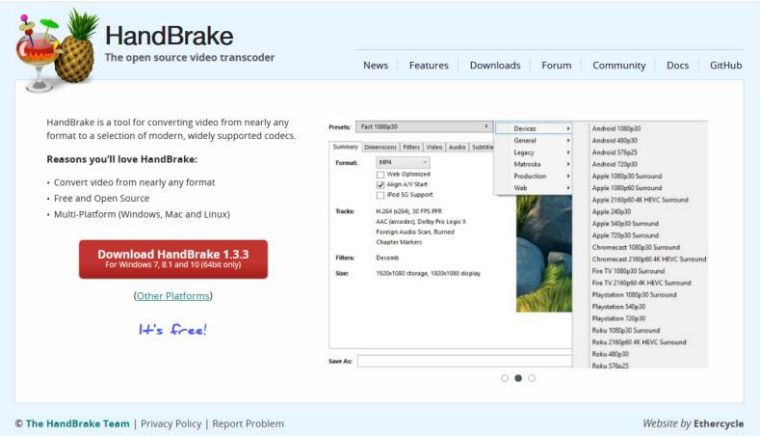
Course Work
Video Submissions

For some tasks we ask for a video explaining your solution

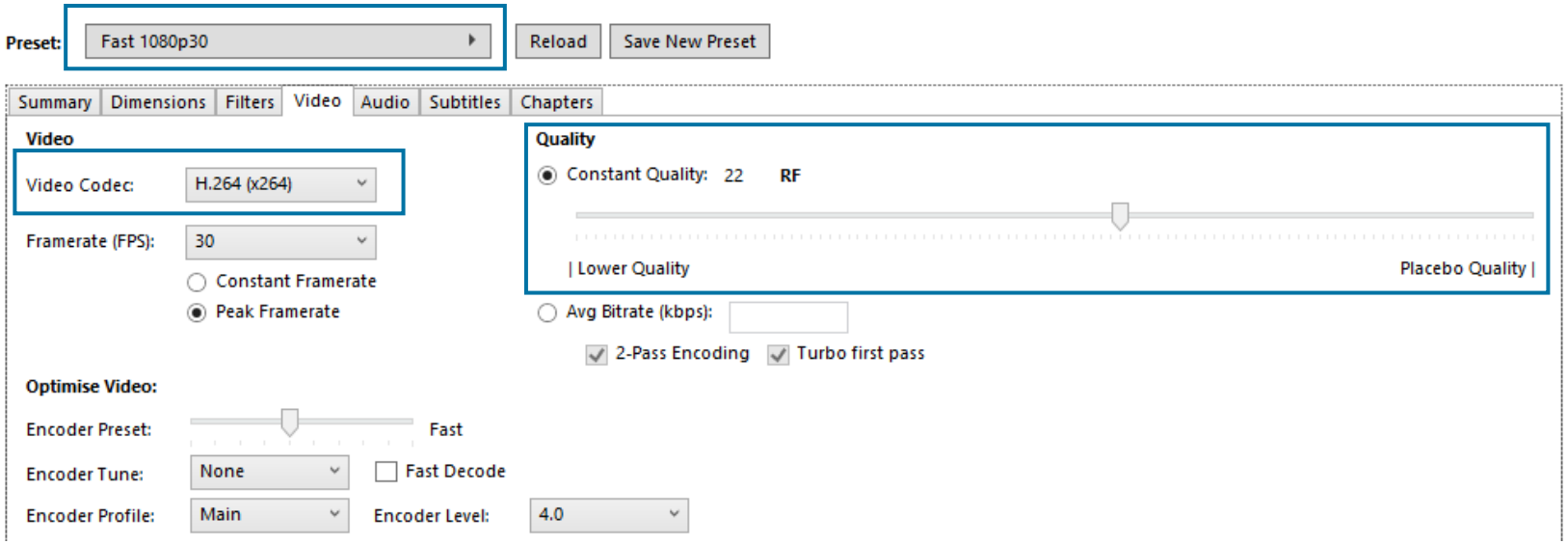
- Depending on task: either screencast or video recording
- You don't have to record yourself on video (only your voice explaining your solution)

Make sure to encode your video properly!

- Use the HandBrake tool (free & open source)
- Submit your videos only as mp4 with h264 codec in full-hd resolution (1080p)
- Adjust the quality that your video is <= 50 MB per minute (1 min = max. 50 MB, 2 min = max. 100 MB, ...)



https://handbrake.fr



Course Work
OPAL

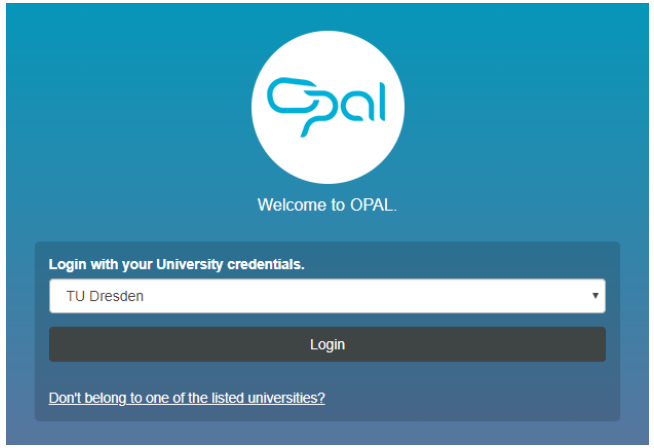
We are using a OPAL for the course organization

- Login with ZIH credentials
- Use the forum for questions!

Material and submissions are handled via OPAL

- Assignments uploaded in folder (tasks and info)
- Notes after the presentation, for recap!

- If you submit your results, do it here, request feedback via the forum or email!



DATA VISUALIZATION

INTERACTIVE MEDIA LAB DRESDEN

Computergraphik und Visualisierung

This lecture covers the basics and practices of scientific visualization of measurement and experimental data, as well as simulation results. It teaches the basics of visual perception and their influence on the design of visualizations. Students will learn to confidently specify data by dimension, feature types, and structure and select appropriate visual attributes for a given specification. They learn the most important visualization forms for 2-, 3- and multidimensional observation spaces, as well as for scalar, vectorial, tensor-valued and multidimensional features. They will be able to select suitable techniques for the respective visualization task.

The course work will make students familiar with basic presentation- and interaction techniques and teaches them to thoroughly implement them in an interactive visual analysis system. It introduces important visualization frameworks and allows students to gather practical experience with them.

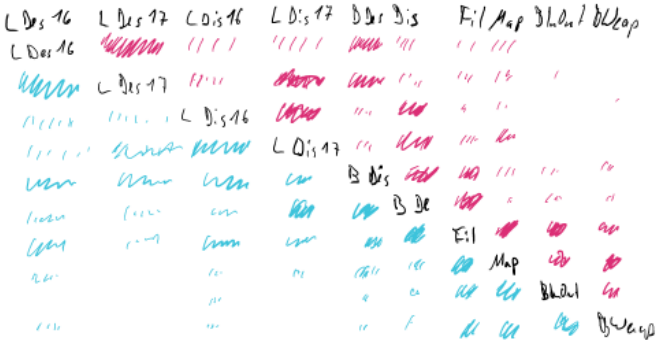
Course Work

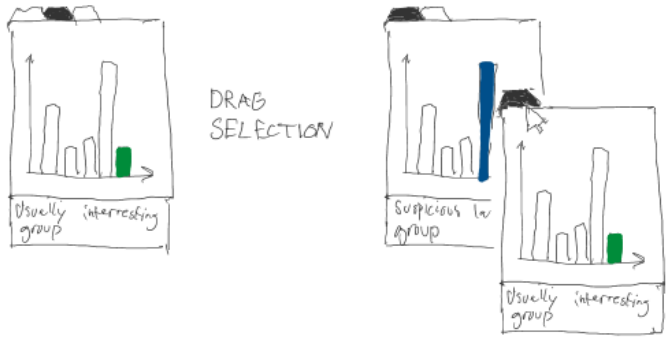
(2) Toolkits

TUD

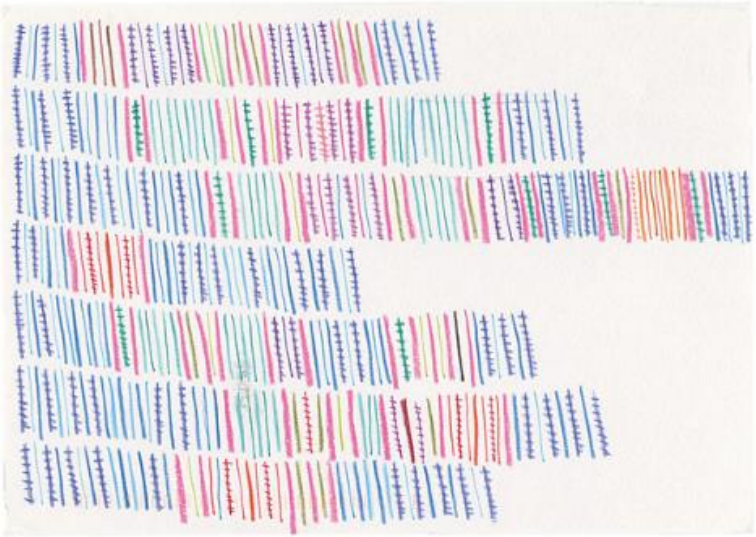
Toolkits
Pen and Paper ;)

- For the first assignment: No visualization toolkit, no programming, no Illustrator (no Paint either)
- But also in future assignments: Draw, scribble, sketch! Use simple tools to sketch first ideas
- Instead of the tools, first think of “what” and “why”!









http://www.dear-data.com/theproject

Toolkits
Web Visualizations – HTML, JavaScript & CSS

The Basics:

- If you don‘t know HTML, JS, and CSS yet – now‘s the time to learn!
- There are a multitude of tutorials available:
  - Best documentation: Mozilla Developer Network (https://developer.mozilla.org/en-US/)
  - JavaScript „Hello World“ Tutorial: https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/JavaScript_basics
  - 33 Concepts Every JavaScript Developer Should Know: https://github.com/leonardomso/33-js-concepts
- Good JavaScript questions:
  - What is the scope of variables in JavaScript?
  - How does the “this” keyword work?
  - Equals operator (== vs ===) in JavaScript comparisons?

```html
<html>
  <head><title>Hello World</title></head>
  <body>
   <h1 id="title">Hello World</h1>
   <p>Quite easy, isn‘t it?</p>
  </body>
</html>
```

```css
#title {
  font-size: 32px;
}
body p {
  background-color: #AEAEAE;
}
```

```javascript
let elem = document.querySelector('#title');
elem.innerHTML = "My New Title";
```

Toolkits
Web Visualizations – D3.js



What‘s D3.js?

- D3 – Data Driven Documents
- JavaScript library for dynamic data visualization
- uses HTML5, SVG, Canvas elements
- Functions to read, select, render, …

- Minimal working example, fills <p> element with numbers

- More examples: http://d3js.org/

```
let p = d3.select("body")
  .selectAll("p")
  .data([4, 8, 15, 16, 23, 42])
  .enter().append("p");
p.text(function(d) { return d; });
```

Toolkits
Web Visualizations – D3.js

The Basics

- Central to D3 are selections:
  - `d3.select("#id")`
  - `d3.selectAll(".class")`
- Other functionalities work on these selections:
  - `d3.select("#id").attr("width", 100);`
  - `d3.select("#id").style("color", "#A2A2A2");`
  - `d3.select("#id").append("p");`
  - `d3.select("#id").remove();`
- D3 supports chained function calls
  - ```
    d3.select("#id")
      .attr("width", 100)
      .style("color", "#A2A2A2");
    ```

Data Binding – D3's superpower

- Associates an array of data items to DOM elements (to an selection)
- This allows:
  - Setting attributes based on data values
  - Adding (entering) corresponding DOM elements

```
let p = d3.select("body")
  .selectAll("p")
  .data([4, 8, 15, 16, 23, 42])
  .enter().append("p");
p.text(function(d) { return d; });
```

  - Removing (exiting) elements
```

Toolkits
Web Visualizations – D3.js

Example: Hover

```
let p = d3.select("body")
  .selectAll("p")
  .data([4, 8, 15, 16, 23, 42])
  .enter().append("p");
p.text(function(d) { return d; })
  .on('mouseover', function(d) {
    d3.select(this).style('background-color', 'blue');
  })
  .on('mouseout', function(d) {
    d3.select(this).style('background-color', 'white');
  });
```

Toolkits
Web Visualizations – D3.js

D3 meets SVG

- SVG (Scalable Vector Graphics) are vector-based graphics
- Similar syntax to HTML; can be included in DOM
- D3 works with all DOM Elements – including SVG

```
let rects = d3.select("svg")
  .selectAll("rect")
      .data([4, 8, 15, 16, 23, 42])
  .enter().append("rect");

rects
  .attr("width", function(d) { return d; });
  .attr("height", 10);
  .attr("x", 0);
  .attr("y", function(d, i) { return i * 15; });
```

Congratulations to our first visualization!

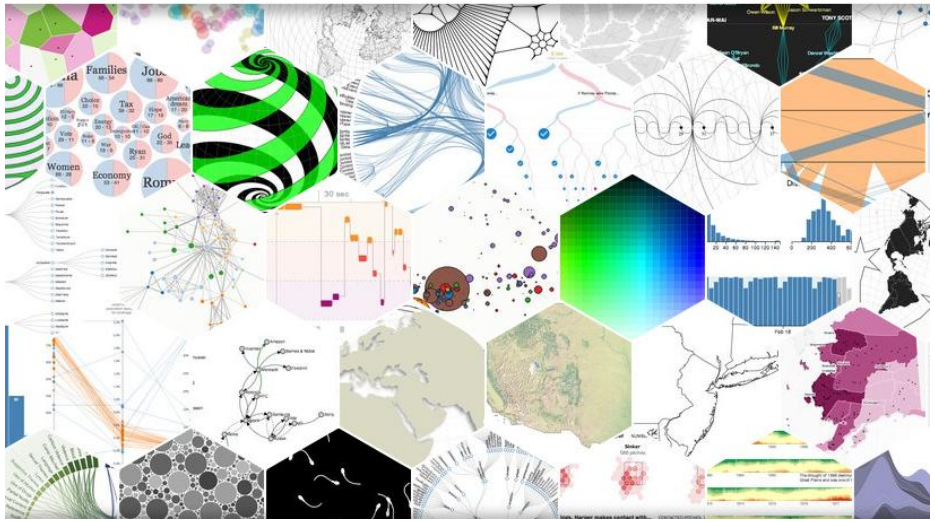


Toolkits
Web Visualizations – D3.js

Further functionalities

- Generating axes & scales
  - ```
    let scaleX = d3.scaleLinear()
      .domain([-100,100]).range([0,500]);
    let axisX = d3.axisBottom();
    axisX.scale(scaleX);
    ```
- Import data from CSV or TSV files
  - ```
    d3.csv("my_data.csv", function (error, data){
      // callback on load
    };
    ```
- Transitions & animations
  - ```
    d3.select("#id").transition()
      .duration(750)
      .attr("width", function(d) { return d * 10 };
    ```

Data-Driven Documents

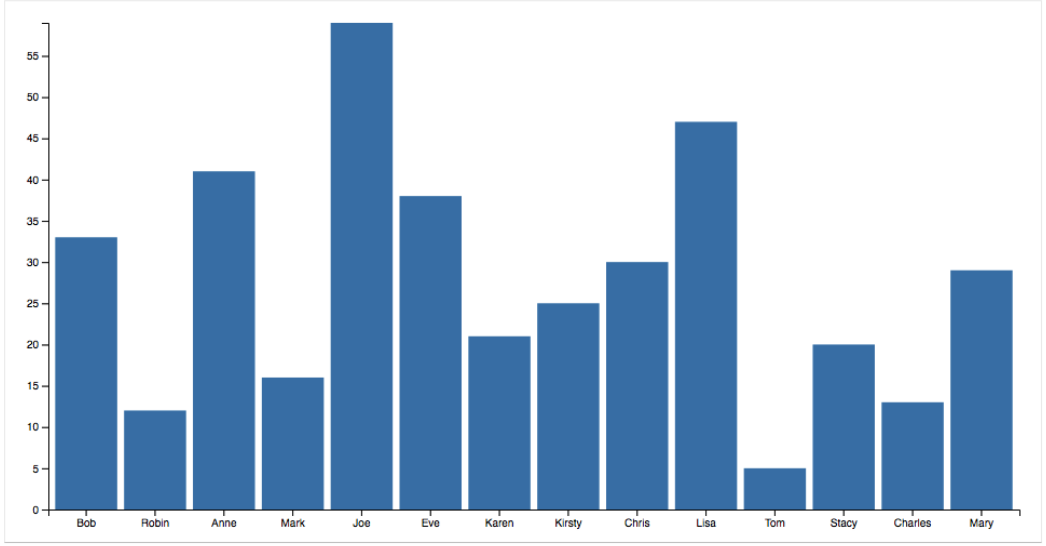


Examples: http://d3js.org/
    ```

Toolkits
Web Visualizations – D3.js

Bar Chart:

https://bl.ocks.org/d3noob/bdf28027e0ce70bd132edc64f1dd7ea4




```
<!DOCTYPE html>
<meta charset="utf-8">
<style> /* set the CSS */

.bar { fill: steelblue; }

</style>
<body>

<!-- load the d3.js library -->
<script src="//d3js.org/d3.v4.min.js"></script>
<script>

// set the dimensions and margins of the graph
var margin = {top: 20, right: 20, bottom: 30, left: 40},
    width = 960 - margin.left - margin.right,
    height = 500 - margin.top - margin.bottom;

// set the ranges
var x = d3.scaleBand()
          .range([0, width])
          .padding(0.1);
var y = d3.scaleLinear()
          .range([height, 0]);

// append the svg object to the body of the page
// append a 'group' element to 'svg'
// moves the 'group' element to the top left margin
var svg = d3.select("body").append("svg")
    .attr("width", width + margin.left + margin.right)
    .attr("height", height + margin.top + margin.bottom)
  .append("g")
    .attr("transform",
          "translate(" + margin.left + "," + margin.top + ")");

// get the data
d3.csv("sales.csv", function(error, data) {
  if (error) throw error;

  // format the data
  data.forEach(function(d) {
    d.sales = +d.sales;
  });

  // Scale the range of the data in the domains
  x.domain(data.map(function(d) { return d.salesperson; }));
  y.domain([0, d3.max(data, function(d) { return d.sales; })]);

  // append the rectangles for the bar chart
  svg.selectAll(".bar")
      .data(data)
    .enter().append("rect")
      .attr("class", "bar")
      .attr("x", function(d) { return x(d.salesperson); })
      .attr("width", x.bandwidth())
      .attr("y", function(d) { return y(d.sales); })
      .attr("height", function(d) { return height - y(d.sales); });

  // add the x Axis
  svg.append("g")
      .attr("transform", "translate(0," + height + ")")
      .call(d3.axisBottom(x));

  // add the y Axis
  svg.append("g")
      .call(d3.axisLeft(y));

});

</script>
</body>
```

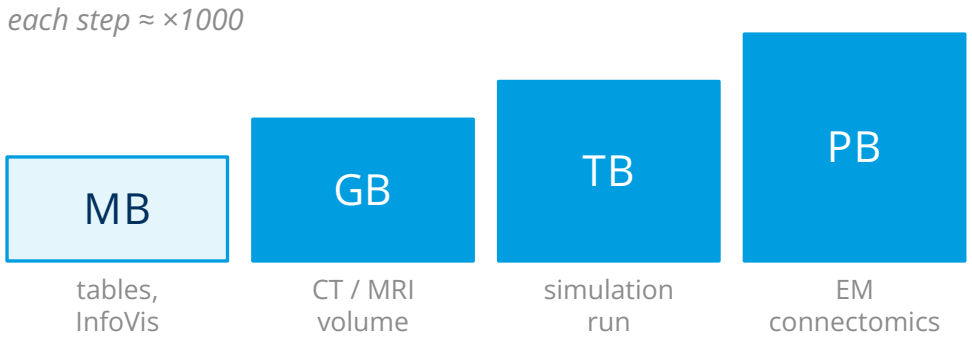
Explanation:

http://www.d3noob.org/2014/02/making-bar-chart-in-d3js.html

Toolkits
Scientific Visualization – Large Data

- SciVis data is typically large
  - CT scan: 512 × 512 × 1000 voxels × 16 bit ≈ 0.5 GB
  - Simulations (climate, fluids): terabytes per run
  - Electron microscopy (connectomics): up to petabytes
- Classic approach: system programming languages with low-level control over memory and parallelism
  - C++ for the application and data handling
  - GPU computing for the heavy lifting: CUDA, OpenCL, compute shaders



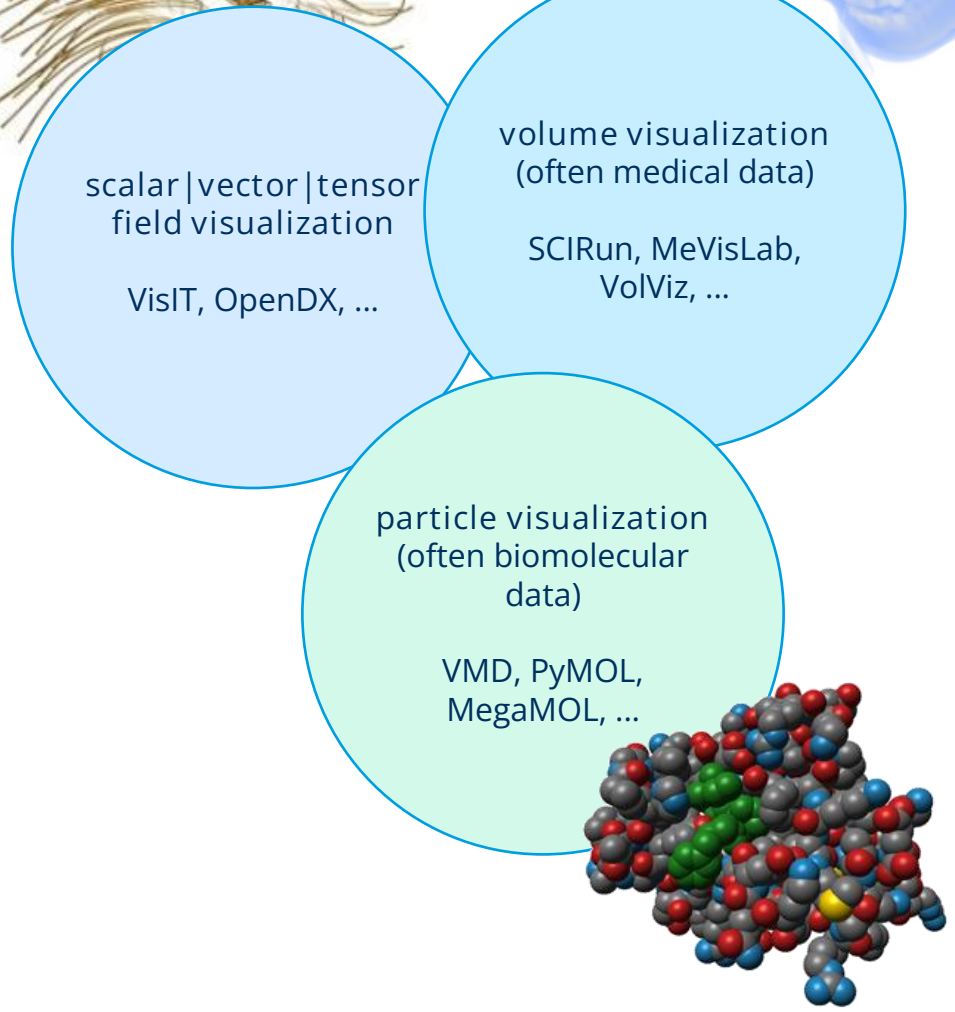


Toolkits
Scientific Visualization – Toolkits

- Many toolkits, most of them domain-specific
- Many are built on the Visualization Toolkit (VTK)
  - C++, 700+ classes, 1M+ lines of code
- ParaView: free VTK-based desktop application
- Python bindings available (e.g. PyVista)

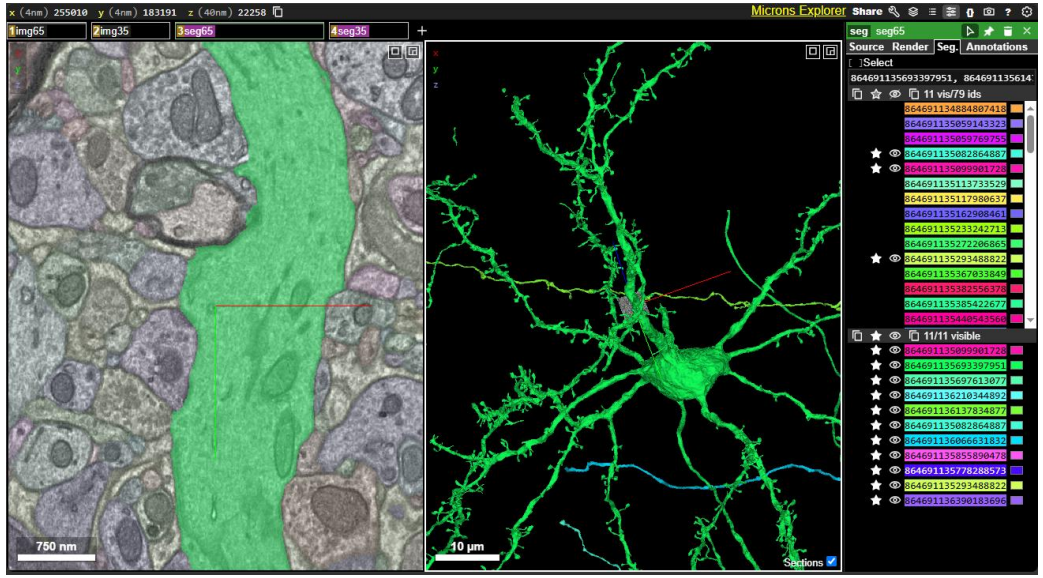




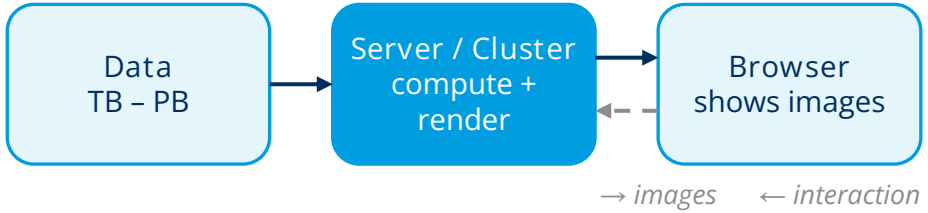


Toolkits
Scientific Visualization – Moving to the Web

- More and more SciVis runs in the browser – no installation
- Rendering in the browser
  - WebGL and, since 2025, WebGPU in all major browsers (compute shaders on the web)
  - e.g. vtk.js – the JavaScript version of VTK
- Remote / distributed rendering
  - Server/cluster renders the large data, browser only shows images
  - e.g. ParaView Web, Kitware trame
- Computation in the backend, interaction in the frontend
  - Python/C++ on the server, visualization in the browser (e.g. Jupyter)
- Streaming huge data on demand
  - e.g. Neuroglancer: petabyte-scale brain EM data in the browser



Neuroglancer: browsing petabyte-scale EM data in the browser (example)

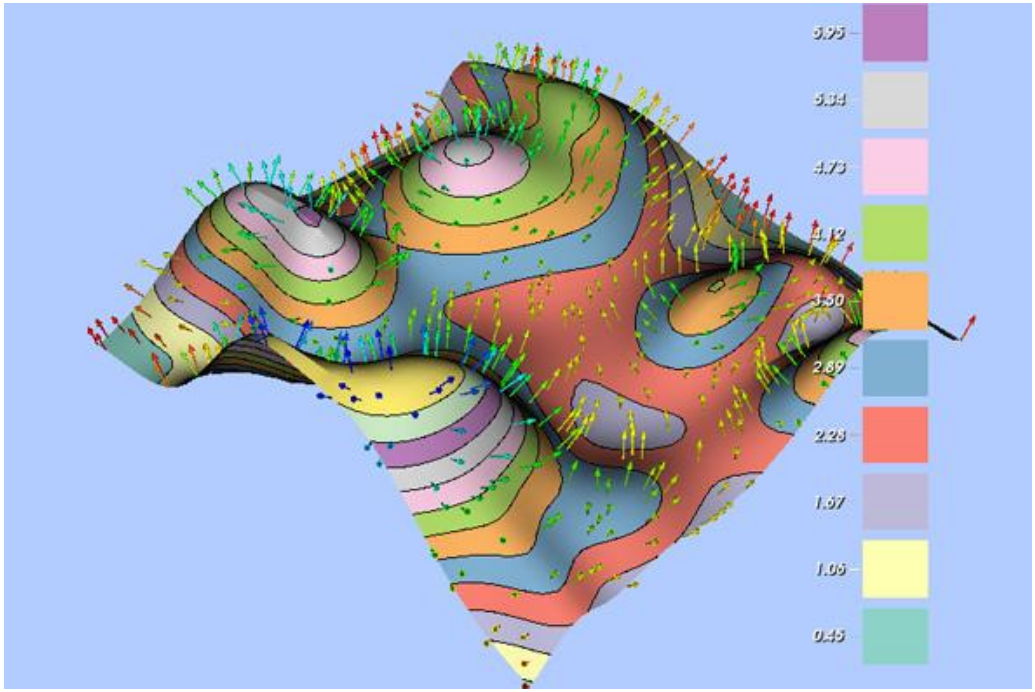


Toolkits
Scientific Visualization – Goals

Goals

- Understand how scalar fields, height fields and volumes become images
- Apply core SciVis techniques from the lecture:
  - Transfer functions and color mapping
  - Contouring (iso-lines / iso-surfaces)
  - Volume rendering
- Understand the effect of visualization parameters and typical artifacts

Toolkit and setup will be announced with Assignment 4



Example: contour lines, elevation bands and normals as glyphs (image: vtk.org)

Toolkits
What's Next?

The toolkits will be used in later assignments!

- Try them out as early as possible
- Discuss issues and challenges with your classmates
- Ask us via mail
- Toolkits for the SciVis part will be introduced with Assignment 4

